

מעבדה 06 — gMSA · Group Managed Service Account

חשבונות שירות מנוהלים אוטומטית · סיסמאות מתחלפות אוטומטית · ללא חשיפת credentials

רמת קושי: ★★ ★ מתקדם

זמן משוער: 30-45 דקות

מודול 5 — ניהול מערכות ושירותים

סיום מודול 5

→ הקודם: DR Documentation

מטרות הלמידה

- להבין את הבעיה בחשבונות שירות מסורתיים (סיסמה קבועה, חשיפה, סיכון)
- ליצור KdsRootKey — הבסיס הקריפטוגרפי ל-gMSAs
- ליצור Group Managed Service Account
- להגדיר אילו מחשבים יכולים לאחזר את הסיסמה האוטומטית
- להתקין ולהריץ שירות (לדוגמה — Scheduled Task) שרץ תחת gMSA
- לאמת שהסיסמה אכן מתחלפת אוטומטית כל 30 יום (במעבדה — 7 שניות)
- להבין מתי gMSA המתאים ומתי לא

למה המעבדה הזו חשובה

בכל ארגון יש שירותים שרצים תחת חשבונות שירות (Service Accounts) — SQL Server, IIS application pools, scheduled tasks, backup jobs. בעבר השיטה הייתה — צור משתמש דומיין רגיל, תן לו "Password never expires", שים את הסיסמה במקומות 30 שונים בקונפיגורציות.

הבעיה: אותה סיסמה יושבת בקבצי קונפיג של אפליקציות, registry, scripts, GPOs. כל אחד מ-100 DBAs בארגון יכול לראות אותה. אם תוקף השיג גישה לקובץ אחד — הוא קיבל גישה לכל הסביבה. וכשרוצים לסובב סיסמה — צריך לעדכן ב-30 מקומות, סדר, חצי משבית. **gMSA** פותר את כל זה: סיסמה אקראית של 240 תווים, מתחלפת אוטומטית כל 30 יום, אף אדם לא יודע אותה. רק המחשבים שאישרתם מראש יכולים לאחזר אותה אוטומטית מ-AD.

הערה: gMSA = הצעד הבא אחרי MSA הישן (Managed Service Account). MSA היה לחשבון לכל מחשב בנפרד. gMSA מאפשר לאותו חשבון לרוץ על מספר מחשבים — לכן "Group".

דרישות מקדימות

- ✓ [מעבדה 00 — הקמת AD](#) הושלמה
- ✓ cyberlab-dc01 + cyberlab-ws01 רצים
- קחו snapshot על שתיהן: **Pre-Lab-06**

חלק א' — KdsRootKey (3 דקות)

KDS = Key Distribution Service. Root Key = המפתח הראשי שמפתחים ממנו את הסיסמאות של ה-gMSAs. בלי הוא — אי אפשר ליצור gMSA.

1 שלב — בדקו אם יש Root Key

היכנסו ל-cyberlab-dc01 כ- **CYBERLAB\Administrator**. PowerShell Admin.

```
Get-KdsRootKey
```

CLI

הערה: אם המסך ריק = אין Root Key. אם יש פלט = יש מפתח קיים, דלגו לחלק ב'.

2 — צרו Root Key (במצב Lab — מיידי)

2

בארגון אמיתי המפתח מתחיל לעבוד 10 שעות אחרי יצירה (כדי שיתפזר לכל ה-DCs). במעבדה — נכפה תאריך אחורה כדי שיהיה זמין מיד:

```
Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10))
```

CLI

✅ **תוצאה צפויה:** פלט מציג GUID של המפתח. `Get-KdsRootKey` מציג עכשיו את המפתח שיצרתם.

⚠️ **שימו לב:** בארגון אמיתי **לא** משתמשים ב-`AddHours(-10)`. זה רק במעבדה. שם רגיל: `Add-KdsRootKey -EffectiveImmediately` (יחכה 10 שעות).

🤖 **עצרו וחשבו:** למה 10 שעות? שני הגנות: (1) זמן ל-AD replication להפיץ את המפתח לכל ה-DCs. (2) שיש זמן לבטל אם נוצר בטעות.

חלק ב' — יצירת gMSA (7 דקות)

1 שלב 3 — צרו קבוצה שתאחסן את המחשבים המאושרים

1

הסיסמה של gMSA זמינה רק למחשבים שהם חברים בקבוצה הזו.

```
New-ADGroup -Name "gMSA-WebAppHosts" `
-GroupScope Global`
```

CLI

```
-GroupCategory Security `
-Path "CN=Users,DC=cyberlab,DC=local" `
-Description "Hosts allowed to retrieve gMSA-WebApp password"
```

✓ **תוצאה צפויה:** הקבוצה נוצרה.

שלב 4 — הוסיפו את cyberlab-ws01 לקבוצה

2

```
CLI
Add-ADGroupMember -Identity "gMSA-WebAppHosts" -Members "cyberlab-
```

✓ **תוצאה צפויה:** השם `cyberlab-ws01$` (עם \$) מציין שזה computer account, לא user. בדיקה:

```
CLI
Get-ADGroupMember -Identity "gMSA-WebAppHosts"
```

✓ **תוצאה צפויה:** תראו `cyberlab-ws01` ברשימה.

שלב 5 — צרו את ה-gMSA

3

```
CLI
New-ADServiceAccount -Name "svcWebApp" `
-DNSHostName "svcWebApp.cyberlab.local" `
-PrincipalsAllowedToRetrieveManagedPassword "gMSA-WebAppHosts"
-ManagedPasswordIntervalInDays 30
```

✓ **תוצאה צפויה:** חשבון השירות נוצר. בדיקה:

```
Get-ADServiceAccount -Identity svcWebApp
```

CLI

✓ **תוצאה צפויה:** פלט מציג `Enabled: True`
`SamAccountName: svcWebApp$`

🙄 **עצרו וחשבו:** הסיסמה אקראית של 240 תווים. אף אחד לא ראה אותה. אף אחד לא יכול. רק המחשבים בקבוצה יכולים לאחזר אותה אוטומטית מ-AD בעת הצורך.

חלק ג' — התקנה ב-cyberlab-ws01 (5 דקות)

1 שלב 6 — אלצו רענון של group membership

חברות בקבוצה נשמרת ב-Kerberos token. צריך טוקן חדש כדי שהמחשב ידע שהוא בקבוצה החדשה. הריצו ב-WS:
 חזרו ל-cyberlab-ws01. הרצו **ריבוט** (לא רק logoff):

```
Restart-Computer -Force
```

CLI

⚠ **שימו לב:** ללא ריבוט — המחשב עדיין חושב שהוא לא בקבוצה. gMSA install ייכשל.

2 שלב 7 — התחברו שוב והתקינו את ה-gMSA

אחרי הריבוט, היכנסו ל-cyberlab-ws01 כ- `CYBERLAB\Administrator`
 :PowerShell Admin

```
Install-ADServiceAccount -Identity svcWebApp
```

CLI

✓ **תוצאה צפויה:** אם אין output = הצלחה.

⚠ **שימו לב:** אם רואים "Cannot install service account" — בדקו: (1) שה-WS חבר בקבוצה, (2) שעשיתם ריבוט, (3) שיש Root Key פעיל.

שלב 8 — אמתו

3

```
Test-ADServiceAccount -Identity svcWebApp
```

CLI

✓ **תוצאה צפויה:** True . ה-gMSA חותקן ופועל על cyberlab-ws01.

חלק ד' — הרצת Scheduled Task תחת ה-gMSA (10 דקות)

נדגים את ה-gMSA בפעולה: ניצור Scheduled Task שרץ תחת חשבון השירות.

שלב 9 — צרו סקריפט בדיקה

1

ב-cyberlab-ws01:

```
New-Item -Path "C:\Scripts" -ItemType Directory -Force
$scriptContent = @"
`$LogFile = 'C:\Scripts\gmsa-test.log'
`$identity = [System.Security.Principal.WindowsIdentity]::GetCurrent
Add-Content ` $LogFile "[`$(Get-Date)] Running as: `$(`$identity.Na
```

CLI

```
"@
$scriptContent | Out-File "C:\Scripts\WhoAmI.ps1" -Encoding UTF8
```

✓ **תוצאה צפויה:** הסקריפט יזהה כל הרצה במי מי הוא רץ.

שלב 10 — צרו Scheduled Task

2

```
CLI
$action = New-ScheduledTaskAction -Execute "powershell.exe" `
    -Argument "-NoProfile -File C:\Scripts\WhoAmI.ps1"

$trigger = New-ScheduledTaskTrigger -At (Get-Date).AddMinutes(1)

$principal = New-ScheduledTaskPrincipal -UserID "CYBERLAB\svcWebApp"
    -LogonType Password

Register-ScheduledTask -TaskName "gMSA-Test" `
    -Action $action `
    -Trigger $trigger `
    -Principal $principal
```

✓ **תוצאה צפויה:** המשימה נרשמת. הסיסמה **לא** נדרשה — Windows יודע לאחזר אותה מ-AD בעת הריצה.

⚠ **שימו לב:** השם `CYBERLAB\svcWebApp$` חובה כולל \$ בסוף. `svcWebApp` בלי \$ = לא יעבוד.

שלב 11 — חכו דקה ובדקו את ה-log

3

```
CLI
Start-Sleep -Seconds 70
```

```
Get-Content C:\Scripts\gmsa-test.log
```

✓ **תוצאה צפויה:** תראו שורה כמו:

```
[2026-04-29 14:35:00] Running as: CYBERLAB\svcWebApp$
```

🙄 **עצרו וחשבו:** המשימה רצה תחת חשבון השירות. אף אחד לא הזין סיסמה. הסיסמה הוחזרה אוטומטית מ-AD בעת ההפעלה. זה הקסם של gMSA.

חלק ה' — אימות שהסיסמה מתחלפת (5 דקות)

סיסמת gMSA מתחלפת אוטומטית כל X ימים (הגדרנו 30). לבדיקה — נציג את ה-metadata.

1 שלב 12 — בדקו את הסיסמה הנוכחית

ב-DC:

```
CLI
Get-ADServiceAccount -Identity svcWebApp -Properties msDS-ManagedPasswordId
```

✓ **תוצאה צפויה:** תראו `PasswordLastSet` = זמן יצירה. ה-`ManagedPasswordId` הוא ה-GUID של המפתח הנוכחי.

2 שלב 13 — Reset יזום (סימולציה של חידוש)

נכריח רענון מידי של הסיסמה:

```
CLI
Reset-ADServiceAccountPassword -Identity svcWebApp
```



```
Get-ADServiceAccount -Identity svcWebApp -Properties PasswordLastSet
```

תוצאה צפויה: **PasswordLastSet** ✓
מציג את הזמן הנוכחי. סיסמה חדשה
אקראית נוצרה. אף אחד לא ראה אותה.

שלב 14 — אמתו שהשירות עדיין עובד

3

חזרו ל-cyberlab-ws01 ובדקו:

```
Test-ADServiceAccount -Identity svcWebApp
```

תוצאה צפויה: **True** ✓
למרות שהסיסמה הוחלפה, ה-WS עדיין יכול לאחזר אותה.

הריצו את ה-Scheduled Task שוב (Start-ScheduledTask) ובדקו ב-log:

```
Start-ScheduledTask -TaskName "gMSA-Test"  
Start-Sleep -Seconds 5  
Get-Content C:\Scripts\gmsa-test.log -Tail 1
```

תוצאה צפויה: השירות רץ. הריצה החדשה ב-log. למרות שהסיסמה הוחלפה — אין downtime, אין צורך לעדכן קונפיג, אין מעורבות אדם.

אתגרי בונוס

🏆 **אתגר בונוס: בונוס 1 — Multiple hosts**. הוסיפו עוד WS לקבוצה `gMSA-WebAppHosts`.
 הריצו `Install-ADServiceAccount` גם שם. עכשיו אותו חשבון רץ על שני מחשבים. השוו ל-
 MSA הישן (חשבון נפרד לכל מחשב).

🏆 **אתגר בונוס: בונוס 2 — IIS App Pool**. אם ה-WS היה IIS Server — אפשר לתת ל-
 Application Pool לרוץ תחת gMSA: IIS Manager → Application Pools → Advanced-ב-
 Settings → Identity → Custom Account `CYBERLAB\svcWebApp$` + סיסמה ריקה. למה
 זה משופר? (תשובה: לא צריך לרענן סיסמה ב-IIS metabase כל 30 יום).

🏆 **אתגר בונוס: בונוס 3 — מתי gMSA לא מתאים?** חשבו על תרחישים בהם gMSA לא יעבוד:
 (תשובה: שירותים עם interactive logon, חשבונות שצריכים גישה לרשת חיצונית עם
 credentials ספציפיים, אפליקציות שמתקבלות על "hardcoded" username/password ולא
 יכולות לעבוד עם service account interface).

🏆 **אתגר בונוס: בונוס 4 — Audit**. בדקו את ה-Event Log של DC. כל ניסיון לאחזר סיסמה
 נרשם: Security → Event 4662. למה זה חשוב לאבטחה? (תשובה: ניטור — אם פתאום מחשב
 אחר מנסה לאחזר את הסיסמה, יש פלילה.)

צ'ק-ליסט אימות

- ☐ מחזיר מפתח `Get-KdsRootKey`
- ☐ קבוצה `gMSA-WebAppHosts` קיימת ומכילה את `cyberlab-ws01$`
- ☐ חשבון `svcWebApp` קיים ב-AD
- ☐ מחזיר True ב-WS `Test-ADServiceAccount svcWebApp`
- ☐ "Scheduled Task" gMSA-Test פעלה תחת `CYBERLAB\svcWebApp$`
- ☐ שינה `PasswordLastSet` `Reset-ADServiceAccountPassword`
- ☐ אחרי reset, השירות עדיין רץ ללא הזנת סיסמה

השוואה: User Service Account vs gMSA

היבט	USER SERVICE ACCOUNT ישן	GMSA
סיסמה	אדם יוצר, אדם יודע	אקראית 240 תווים, אף אחד לא יודע
החלפה	ידנית, נדירה (אם בכלל)	אוטומטית כל 30 יום
חשיפה	בקובצי קונפיג, GPO, scripts	אף פעם לא מחוץ ל-AD
מספר מחשבים	אחד או ידני בכל מחשב	קבוצת מחשבים, אוטומטי
Audit trail	קשה לעקוב	Event 4662 על כל אחזור
תאימות	כל אפליקציה	רק שירותים שתומכים

עיקרי הדברים

- חשבונות שירות עם סיסמה ידנית = פצצה מתקתקת. הסיסמה זולפת בסופו של דבר.
- gMSA = סיסמה אקראית, אוטומטית, בלתי-נראית. הפתרון המודרני.
- KdsRootKey = המפתח הראשי. צריך פעם אחת לפני יצירת gMSA ראשון.
- גישה למפתח ע"י AD group — רק חברי הקבוצה יכולים לאחזר.
- השירות צריך לתמוך ב-gMSA. רוב המודרניים תומכים. ישנים יותר — לא.
- אחרי reset/rotation — אין downtime, השירות ממשיך לרוץ. זה הקסם.
- Audit ב-Event 4662 — תקיפה תופסת בולטת.

✓ **תוצאה צפויה:** סיימתם את מודול 5 בשלמות. כל 6 המעבדות. אתם יודעים עכשיו את כל הבסיס של ניהול תשתית Windows מודרנית.

בסוף המעבדה ובסוף המודול

1. קחו snapshot על שתי ה-VMs: **End-of-Module-5**
2. השוו את snapshot זה למצב התחלתי **Clean-Install-Pre-AD** — הקצב כמה גדל הסביבה.
3. כיבוי תקין.

💡 **טיפ:** המעבדה הבאה במודול 6 (אבטחת סייבר) תשתמש בסביבה הזו כבסיס. שמרו!

סיום מודול 5 ✓

→ הקודם: DR Documentation

© CyberLabs · מודול 5 — ניהול מערכות ושירותים · מעבדה 06 · 29/04/2026